황민영 Platform Engineer

Name
황민영

Email
yakc0202@gmail.com

SUMMARY

3년 이상 금융권 인프라 운영 업무를 수행했으며 2024년 9월부터 Kubernetes(EKS) 운영을 담당했습니다.
운영 EKS 업그레이드, 장애 대응, 배포 환경과 모니터링 개선 경험을 소개합니다.

메가존클라우드 2023.05 ~ 재직 중 고객사 프로젝트 단위 상주 – C금융그룹 > B핀테크 > A카드

01	운영 EKS 클러스터 · 노드그룹 업그레이드	A카드 · 운영 2개 포함 클러스터 4개 · 노드그룹 5개	p. 3
02	보안 계정 인프라 Terraform 이관	A카드 ·EC2 · EKS 총 14개 코드 관리 편입	p. 4
03	노드 연쇄 장애 대응	B핀테크 · 메모리 Request 개선과 Helm 기반 설정 중앙 관리	p. 5
04	Istio의 MSK 트래픽 식별 및 알람 분리	B핀테크 · 보안 패치 중 반복된 알람 개선	p. 6
05	Argo CD 테스트 환경 분리 및 설정 관리 표준화	B핀테크 · Frontend 테스트 환경 구축과 Helm values.yaml 적용	p. 7
06	모니터링 스택 전환	C금융그룹 · Prometheus 전환과 Exporter 표준화	p. 8
07	추가 문제 해결 사례	A카드 · RDS 비밀번호의 tfstate 기록 방지 A카드 · AWS Health 알람의 계정 정보 수정 B핀테크 · GHES 복제 · 백업 실패	p. 9

01 · A카드

운영 EKS 클러스터 · 노드그룹 업그레이드

시작 상태

컨트롤 플레인 1.33
노드그룹 1.31

제약

컨트롤 플레인은 한 마이너씩
노드는 세 마이너 차이까지 지원

컨트롤 플레인 1.34

1.33 → 1.34
노드그룹 1.31 → 1.34 교체

컨트롤 플레인 1.35

1.34 → 1.35
노드 1.34는 한 단계 차이

노드그룹 1.35

고객사 요청으로 버전 일치
1.34 → 1.35 교체

상황

컨트롤 플레인 1.33, 노드그룹 1.31로 버전 차이가 있었습니다.
대상은 개발 2개 · 운영 2개 클러스터와 노드그룹 5개였습니다.

EKS 1.33의 표준 지원 종료일인 2026년 7월 29일 전에 업그레이드를 완료하도록 계획했으며 목표 버전은 1.35로 정했습니다.

판단

컨트롤 플레인의 순차 업그레이드와 노드 버전 차이 제약을 고려했습니다.
그래서 1.34 단계에서 노드그룹을 먼저 1.34로 맞추는 순서를 정했습니다.

운영 노드의 추가 교체를 줄이기 위해 컨트롤 플레인 1.35 · 노드그룹 1.34 구성을 제안했으나, 고객사의 버전 일치 요구를 반영해 최종 목표를 모두 1.35로 정했습니다.

결과

클러스터 4개와 노드그룹 5개를 1.35로 업그레이드하고 addon을 호환 버전으로 함께 갱신했습니다.

Bastion에서 노드 롤링 교체와 파드 재배치 상태를 확인했습니다.
EKS 1.33 표준 지원 종료 전에 작업을 완료했습니다.

02 · A카드

보안 계정 인프라 Terraform 이관

기준

사고가 나면 영향이 계정 전체로 가는 자원인가

이관

EC2 · EKS

제외

VPC · Subnet · 라우팅

콘솔에 있던 EC2

이름	타입	서브넷
app-a	t3.medium	subnet-a
app-b	t3.large	subnet-b
app-c	t3.medium	subnet-c
app-d	t3.small	subnet-a
...		

· Terraform 코드로 관리되지 않음

· 인스턴스마다 타입과 배치가 다름

먼저 작성한 공통 모듈 → import로 State 연결

```
modules/ec2/
  main.tf
  variables.tf
  outputs.tf

module "app" {
  source    = "../../modules/ec2"
  for_each = local.instances

  instance_type = each.value.type
}
```

상황

Splunk는 Terraform으로 구성하려 했지만 같은 계정 내의 다른 자원은 사람이 콘솔로 만들어 둔 상태였습니다.

이대로 가면 관리 포인트가 콘솔과 코드 두 갈래로 나뉘는데 고객사도 이 부분을 고민하고 있었습니다.

→ Splunk 도입을 계기로 기존 자원도 Terraform 관리 대상으로 편입하자고 제안했습니다.

판단

변경 영향과 복구 부담이 큰 VPC · Subnet · 라우팅은 기존 관리 방식을 유지했습니다.

이번 이관 범위를 EC2 · EKS로 한정했습니다.

공통 구성과 인스턴스별 차이를 나누어 관리하도록 모듈 구조를 정했습니다.

변경 위험이 낮은 자원부터 순차적으로 이관하기로 했습니다.

결과

EC2 · EKS 총 14개를 Terraform 관리 대상으로 편입했습니다.

기존 운영 설정이 변경되지 않도록 코드와 실제 설정을 대조하고 plan에서 변경 사항이 없음을 확인했습니다.

VPC · Subnet · 라우팅은 기존 관리 방식을 유지했습니다.

03 · B핀테크

노드 연쇄 장애 대응

근본 원인

모든 파드에 낮은 Request 메모리를 일괄 적용
노드에 여유가 있다고 계산돼 파드가 계속 배치

배포 누락

컨테이너 메모리 설정 추가만 배포되고 파드 Request 조정 누락

확산

연쇄 노드 장애와
파드 재스케줄링 반복
ASG 조정으로도 멈추지 않음

2차 장애

Docker Hub rate limit으로
Istio 이미지 pull까지 실패

복구

ECR에 준비해둔 이미지로 즉시 전환
파드 Request 수정 후 재배포

상황

반복되는 노드 메모리 알람을 줄이기 위해 JVM 힙 설정 추가와 파드 requests 조정을 함께 진행했습니다.
적용 과정에서 힙 설정만 반영되고 requests 조정이 누락돼 노드 메모리 부족과 파드 재스케줄링이 반복됐습니다.

판단

ASG 조정 후에도 재스케줄링이 반복됐습니다. 노드 수 조정보다 누락된 메모리 Request 수정이 우선이라고 판단했습니다.

Istio 이미지 pull 실패에는, 기존에 ECR로 옮겨둔 이미지를 활용해 Docker Hub를 거치지 않고 복구하는 방식을 선택했습니다.

같은 설정 누락을 줄이려면 관리 방식도 바꿔야 한다고 봤습니다. 파드 자원 설정을 Helm으로 중앙 관리하기로 했습니다.

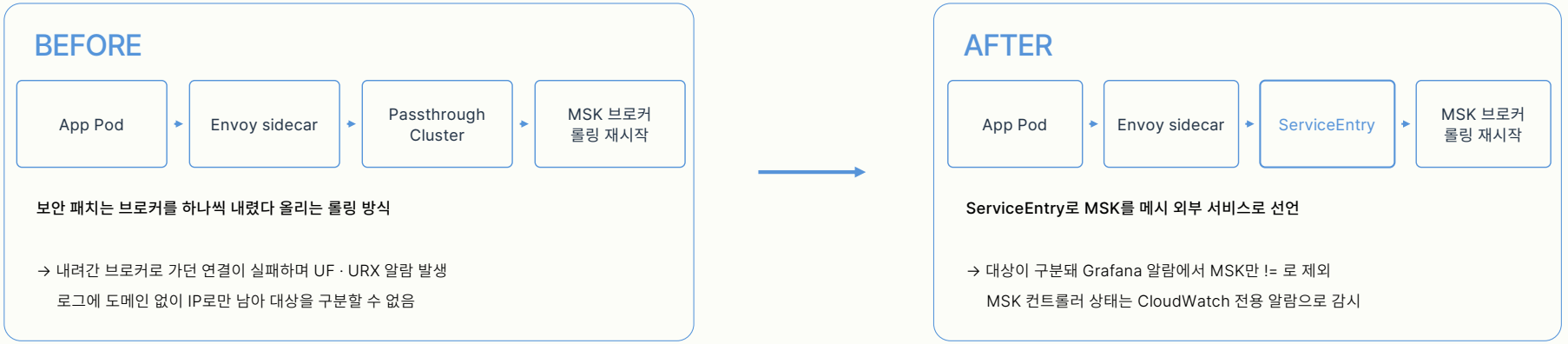
결과

ECR 이미지로 전환해 Istio 이미지 pull 실패를 해소하고 파드 Request 수정으로 안정화했습니다.

파드 자원 설정을 Helm으로 중앙 관리했습니다. 메모리 설정을 최적화하고 노드 스펙 상향을 진행했습니다.
개발팀과 배포 절차 · 커뮤니케이션 체계도 정비했습니다.

04 · B핀테크

Istio의 MSK 트래픽 식별 및 알람 분리



상황

MSK 보안 패치마다 Istio 오탐 알람이 반복됐습니다.

기존 Support 기록대로 토픽 설정을 조정했지만 알람은 그대로였습니다.

재문의 끝에 브로커가 하나씩 재시작되는 롤링 패치임을 확인했지만 정기 작업과 실제 장애를 구분할 방법이 없었습니다.

Envoy 로그를 보니 목적지가 IP로만 찍혔고 브로커 IP와 대조해 MSK 통신임을 확인했습니다.

판단

다른 외부 통신의 장애 감지를 유지하려면 Istio 알람 전체를 끄지 않고 MSK 트래픽만 식별해 분리해야 한다고 판단했습니다.

MSK를 구분할 수 있도록 ServiceEntry를 적용하기로 했습니다.
Grafana 알람에서는 해당 대상만 제외하는 방식을 선택했습니다.
MSK 컨트롤러 상태는 CloudWatch 전용 알람으로 별도 감시하기로 했습니다.

결과

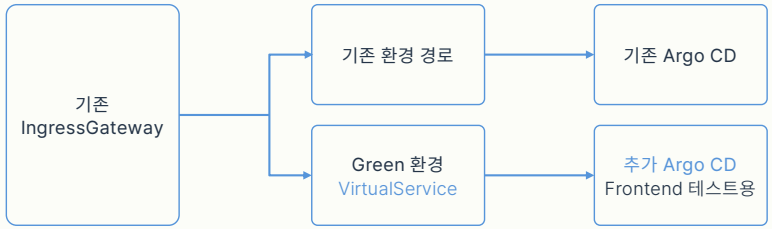
ServiceEntry 적용 후 Grafana 알람 쿼리에서 MSK를 제외해 보안 패치 때 반복되던 Istio 오탐 알람을 제거했습니다. 다른 외부 통신의 UF · URX 감지는 유지했습니다.

공식 문서와 Support 검증을 거쳐 ActiveControllerCount 기반으로 MSK 컨트롤러 상태를 감시하는 CloudWatch 전용 알람을 구성했습니다.

05 · B핀테크

Argo CD 테스트 환경 분리 및 설정 관리 표준화

테스트 환경 분리



기존 클러스터 IngressGateway 재사용, 환경별 Ingress 구성

Argo CD 설정 관리 표준화

구분	변경 전	변경 후
Frontend	Terraform locals	Helm values.yaml
Backend	Helm values.yaml	Helm values.yaml

Frontend · Backend 설정 방식 통일

Frontend Argo CD에 GitHub SSO 적용

상황

Frontend 팀에서 테스트 환경 확장을 요청했습니다.
기존 환경과 같은 구성을 유지하면서 독립적으로 사용할 Argo CD 인스턴스가 필요했습니다.

Argo CD 구성에 사용하는 Helm 설정 값을 Frontend는 Terraform locals에, Backend는 별도 values.yaml 파일에 정의하고 있어 설정 관리 방식이 달랐습니다.

판단

기존 환경과 같은 구성을 유지하면서 Argo CD를 분리하는 요구였으므로, 클러스터를 새로 구성하기보다 기존 클러스터와 IngressGateway를 활용하는 쪽을 택했습니다.

Argo CD 설정 값은 Frontend · Backend가 같은 방식으로 관리할 수 있도록 Backend에서 사용하던 별도 values.yaml 파일 기준으로 통일하기로 했습니다.

결과

테스트용 Argo CD를 구성하고 접속과 로그인, 동작을 확인했습니다.

Frontend 설정을 terraform locals에서 values.yaml로 옮겨 Backend와 관리 형식을 통일하고 GitHub SSO를 적용했습니다.

이후 권한 설정을 변경할 때도 Backend와 동일하게 values.yaml만 수정하면 됐습니다.

06 · C금융그룹

모니터링 스택 전환

BEFORE

CloudWatch 데이터소스

계열사 계정 → CloudWatch → Grafana

지표를 조회할 때마다 비용이 발생하는 구조

- 조회량 증가에 따른 CloudWatch API 비용 부담
- Python · crontab 커스텀 메트릭을 직접 유지보수
- 계열사마다 수집 방식이 상이

AFTER

Grafana Agent · Prometheus

Grafana Agent → VPC Endpoint → Prometheus → Grafana

수집한 지표를 Prometheus에 저장해 조회 비용과 분리

- 다계정 환경에서 VPC Endpoint 기반 보안 검증 수행
- Blackbox Exporter로 대체 가능한 지표는 표준 Exporter로 이관
- 계열사 간 수집 방식 통일

상황

Grafana에서 CloudWatch 지표를 조회하는 구조여서 조회 비용이 발생했습니다.

Grafana가 올라간 EC2에서 crontab과 Python 파일로 커스텀 메트릭을 직접 만들어 운영하고 있었습니다.

판단

대시보드와 알람을 다시 구축하는 부담이 있어도 CloudWatch 지표 조회 비용을 줄이기 위해 수집 구조 전환이 필요하다고 판단했습니다.

각 계열사도 이미 Grafana Agent · Prometheus 기반이었습니다.

은행도 같은 방식으로 전환해 세 곳의 유지보수 방식을 통일하기로 했습니다.

결과

Python · crontab 기반 커스텀 메트릭 중 대체 가능한 항목을 Grafana Agent의 Blackbox 통합 기능으로 전환해 코드 유지 보수 범위를 줄였습니다.

Prometheus 기반으로 옮기면서 대시보드와 알람을 새로 구축했습니다.

모니터링 수집 구조 전환에 참여해 모니터링 비용을 전월 대비 약 13% 절감하는 데 기여했습니다.

07 · 추가 문제 해결 사례

GHES 복제 · 백업 실패

GHES 업그레이드 이후 복제와 백업이 정상 동작하지 않았습니다. 복제본 EC2를 재생성해도 같은 증상이 발생해 원본 서버의 연결 설정까지 조사 범위를 넓혔습니다.
SSH 디버그 로그와 키 · 디렉터리 권한을 점검했습니다. 원본 서버의 권한 문제를 확인하고 권한 수정으로 복제 · 백업을 정상화했습니다.
진단 절차와 해결 방법은 문서화해 팀에 공유했습니다.

RDS 비밀번호의 tfstate 기록 방지

인계받은 코드의 tfvars, tfstate, 애플리케이션 설정에서 RDS 마스터 비밀번호가 관리되고 있었습니다.
Ephemeral 변수와 password_wo를 적용해 비밀번호가 state에 기록되지 않도록 변경했습니다.
애플리케이션 설정은 담당 부서에 요청해 수정됐으며 tfvars의 비밀번호를 Secrets Manager 기반으로 전환하는 작업은 후속 계획으로 구분했습니다.

AWS Health 알람의 계정 정보 수정

서울 · 버지니아 리전에 EventBridge를 구성하고 SNS · Lambda를 통해 AWS Health 알람을 전달했습니다.
구축 후 검증에서 알람의 Account ID가 실제 이벤트 발생 계정이 아닌 중앙 수신 계정으로 표시되는 문제를 발견했습니다.
Lambda 로직을 수정해 이벤트 발생 계정이 표시되도록 개선했습니다.